

Q1(a)	<pre> # Program to find factorial using for loop a = int(input("Enter a number: ")) fact = 1 for i in range(1, a + 1): fact *= i print("Factorial =", fact) </pre>
Q1(b)	<pre> # Program to check Neon Number num = int(input("Enter a number: ")) square = num * num sum_digits = 0 # Find sum of digits of square while square > 0: digit = square % 10 sum_digits += digit square //= 10 # Check Neon condition if sum_digits == num: print(num, "is a Neon Number") else: print(num, "is not a Neon Number") </pre>
Q2(a)	<pre> # Program to copy contents from source.txt to destination.txt # Step 1: Open source file in read mode f1 = open("source.txt", "r") # Step 2: Read data from source file data = f1.read() # Step 3: Open destination file in write mode f2 = open("destination.txt", "w") # Step 4: Write data into destination file f2.write(data) # Step 5: Close both files f1.close() f2.close() print("File copied successfully!") </pre>
Q2(b)	<pre> f = open("data.txt", "r") line = f.readline() </pre>

	<pre> while line != "": pos = f.tell() - len(line) # starting position of line if "Python" in line: print("Position:", pos) print(line, end="") line = f.readline() f.close() </pre>												
Q3	i) for loop vs while loop ◆ Differences: <table> <thead> <tr> <th>For Loop</th><th>While Loop</th></tr> </thead> <tbody> <tr> <td>Used when number of iterations is known</td><td>Used when condition-based repetition</td></tr> <tr> <td>Initialization, condition, increment in one line</td><td>Initialization, condition, increment written separately</td></tr> </tbody> </table> ◆ Syntax: <pre> # for loop for i in range(1, 6): print(i) # while loop i = 1 while i <= 5: print(i) i += 1 </pre> ii) break VS continue ◆ Differences: <table> <thead> <tr> <th>Break</th><th>continue</th></tr> </thead> <tbody> <tr> <td>Terminates the loop completely</td><td>Skips current iteration and continues loop</td></tr> <tr> <td>Control goes outside the loop</td><td>Control goes to next iteration</td></tr> </tbody> </table> ◆ Example: <pre> # break example for i in range(1, 6): if i == 3: break print(i) # Output: 1 2 </pre>	For Loop	While Loop	Used when number of iterations is known	Used when condition-based repetition	Initialization, condition, increment in one line	Initialization, condition, increment written separately	Break	continue	Terminates the loop completely	Skips current iteration and continues loop	Control goes outside the loop	Control goes to next iteration
For Loop	While Loop												
Used when number of iterations is known	Used when condition-based repetition												
Initialization, condition, increment in one line	Initialization, condition, increment written separately												
Break	continue												
Terminates the loop completely	Skips current iteration and continues loop												
Control goes outside the loop	Control goes to next iteration												

```
# continue example
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
# Output: 1 2 4 5
```

iii) Compiler vs Interpreter

◆ Differences:

Compiler	Interpreter
Translates whole program at once	Translates line by line
Faster execution after compilation	Slower execution

◆ Examples:

- **Compiler:** C, C++
- **Interpreter:** Python

Q4:

- a) **Write a Python program that asks the user to enter two numbers and performs division.** Handle the `ZeroDivisionError` exception and display an appropriate message if the user attempts to divide a number by zero.

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
try:
    print("Result:", a / b)
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

- b) **Write a Python program to create a user-defined exception named `InvalidMarksError`.** Raise this exception if the marks entered by the user are less than 0 or greater than 100. Display an appropriate error message.

```
# Define user-defined exception
class InvalidMarksError(Exception):
    Pass
marks = int(input("Enter marks: "))
if marks < 0 or marks > 100:
    raise InvalidMarksError("Marks should be
between 0 and 100")
print("Valid marks entered:", marks)
```

Q5:

Differentiate between the following pairs by writing any two valid differences for each:

- a) Compile time errors and Run time errors

Feature	Compile Time Errors	Run Time Errors
Occurrence	Occur before execution (during compilation)	Occur during program execution
Cause	Syntax mistakes (e.g., missing :)	Logical issues (e.g., division by zero)

b) Pickling and Unpickling

Feature	Pickling	Unpickling
Definition	Converting Python object into byte stream	Converting byte stream back to object
Purpose	Used to store data in file	Used to retrieve original data

c) Mutable and Immutable objects

Feature	Mutable Objects	Immutable Objects
Modification	Can be changed after creation	Cannot be changed after creation
Example	List, Dictionary	Tuple, String

Q6:

- a) **Write a Python program to enter the names of employees and their salaries from the user and store them in a dictionary. Display the dictionary at the end.**

```
emp = {}
n = int(input("Enter number of employees: "))
for i in range(n):
    name = input("Enter name: ")
    salary = int(input("Enter salary: "))
    emp[name] = salary
print("Employee Dictionary:", emp)
```

- b) **Write a Python program to create a tuple of five numbers from user and display the largest number from the tuple.**

```
nums = []
for i in range(5):
    nums.append(int(input("Enter number: ")))
t = tuple(nums)
print("Tuple:", t)
print("Largest number:", max(t))
```

- c) **Write a Python program to enter a string and count the number of vowels in it.**

```
s = input("Enter a string: ")
count = 0
for ch in s:
    if ch.lower() in 'aeiou':
        count += 1
print("Number of vowels:", count)
```